Практическое занятие №2. Средство мониторинга Zabbix

Цель работы

Изучить архитектуру и возможности системы мониторинга Zabbix. Освоить развёртывание Zabbix Server, веб-интерфейса, СУБД PostgreSQL и Zabbix Agent с помощью Docker Compose. Получить практические навыки работы с веб-интерфейсом Zabbix, создания шаблона мониторинга, добавления хоста и привязки шаблона к хосту.

Оборудование и программное обеспечение

- Виртуальная машина или физический компьютер с **Debian 12**
- Установленные **Docker Engine** и **Docker Compose** (плагин `docker compose`)
- Доступ к интернету для загрузки Docker-образов
- Браузер для доступа к веб-интерфейсу Zabbix
- Минимальные ресурсы: 2 CPU, 4 ГБ ОЗУ, 10 ГБ свободного места на диске

Краткие теоретические сведения

Zabbix — это свободная система мониторинга сетевых сервисов, серверов и сетевого оборудования. Она собирает метрики (загрузка CPU, использование памяти, состояние дисков, доступность сервисов), хранит их в базе данных, строит графики и отправляет оповещения при отклонении от нормы.

Ключевые понятия:

- **Хост (Host)** — устройство, которое отслеживается. Для каждого хоста задаются интерфейсы (Agent, SNMP, IPMI, JMX).
- **Элемент данных (Item)** — конкретный показатель, который собирается с хоста (например, `system.cpu.load` — загрузка процессора).
- **Триггер (Trigger)** — логическое выражение, которое срабатывает при выполнении условия (например, «загрузка CPU выше 2 в течение 3 минут»).
- **Шаблон (Template)** — набор элементов данных, триггеров, графиков и правил автоматического обнаружения. Шаблон можно привязать к множеству хостов, обеспечивая единообразный мониторинг.
- **Группа хостов (Host group)** — логическая группировка хостов для удобства управления.

Архитектура развёртывания в Docker:

Компонент	Docker-образ	Назначение
PostgreSQL	`postgres:15`	Хранение конфигурации и истории метрик
Zabbix Server	`zabbix/zabbix-server-pgsql`	Обработка данных, вычисление триггеров
Zabbix Web (Nginx)	`zabbix/zabbix-web-nginx-pgsql`	Веб-интерфейс пользователя
Zabbix Agent 2	`zabbix/zabbix-agent2`	Сбор метрик с узла

Порядок выполнения работы

1. Подготовка окружения

1. Убедитесь, что Docker и Docker Compose установлены:

```
docker --version
docker compose version
```

2. Создайте рабочую директорию для проекта:

```
mkdir -p ~/zabbix-lab && cd ~/zabbix-lab
```

4.2. Составление файла `compose.yaml`

Создайте файл `compose.yaml` в рабочей директории со следующим содержимым:

```
services:
  postgres:
    image: postgres:15
    container_name: zabbix-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: zabbix
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbix_pwd
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
```

```
      - zabbix-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U zabbix -d zabbix"]
      interval: 10s
      timeout: 5s
      retries: 5

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:7.0-ubuntu-latest
    container_name: zabbix-server
    restart: unless-stopped
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      DB_SERVER_HOST: postgres
      POSTGRES_DB: zabbix
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbix_pwd
    ports:
      - "10051:10051"
    networks:
      - zabbix-net

  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:7.0-ubuntu-latest
    container_name: zabbix-web
    restart: unless-stopped
    depends_on:
      - postgres
      - zabbix-server
    environment:
      ZBX_SERVER_HOST: zabbix-server
      DB_SERVER_HOST: postgres
      POSTGRES_DB: zabbix
      POSTGRES_USER: zabbix
      POSTGRES_PASSWORD: zabbix_pwd
      PHP_TZ: Europe/Moscow
    ports:
      - "80:8080"
    networks:
```

```
      - zabbix-net

  zabbix-agent:
    image: zabbix/zabbix-agent2:7.0-ubuntu-latest
    container_name: zabbix-agent
    restart: unless-stopped
    depends_on:
      - zabbix-server
    environment:
      ZBX_HOSTNAME: "Zabbix server"
      ZBX_SERVER_HOST: zabbix-server
      ZBX_SERVER_PORT: 10051
    ports:
      - "10050:10050"
    networks:
      - zabbix-net

volumes:
  postgres-data:

networks:
  zabbix-net:
    driver: bridge
```

Пояснения к ключевым параметрам:

- `DB_SERVER_HOST`, `POSTGRES_*` — параметры подключения Zabbix Server и Web к PostgreSQL.
- `ZBX_SERVER_HOST` — адрес Zabbix Server для веб-интерфейса и агента (внутри Docker-сети используется имя сервиса `zabbix-server`).
- `ZBX_HOSTNAME` — имя хоста, которое агент сообщает серверу. Значение `"Zabbix server"` соответствует стандартному имени хоста в Zabbix, что упростит привязку шаблона.
- `ports` — проброс портов на хост-машину. Веб-интерфейс будет доступен на порту **80**, Zabbix Server — на **10051**, агент — на **10050**.
- `PHP_TZ` — часовой пояс для корректного отображения времени в веб-интерфейсе.

3. Запуск стека

1. Запустите контейнеры в фоновом режиме:

```
docker compose up -d
```

2. Дождитесь инициализации базы данных (обычно 1–3 минуты). Проверьте статус контейнеров:

```
docker compose ps
```

Все контейнеры должны иметь статус `running` (или `healthy` для `postgres`).

3. При необходимости просмотрите логи Zabbix Server:

```
docker compose logs zabbix-server
```

4. Вход в веб-интерфейс

1. Откройте браузер и перейдите по адресу `http://<IP-адрес_виртуальной_машины>/` (или `http://localhost/`, если работаете локально).
2. Введите учётные данные по умолчанию:
   - **Имя пользователя:** `Admin`
   - **Пароль:** `zabbix`
3. После первого входа **смените пароль** администратора (Administration → Users → Admin → Change password).

5. Изучение интерфейса Zabbix

Ознакомьтесь с основными разделами главного меню:

Раздел	Назначение
Monitoring → Dashboard	Сводная панель с виджетами
Monitoring → Problems	Текущие проблемы (сработавшие триггеры)
Monitoring → Hosts	Список хостов и их доступность
Monitoring → Latest data	Последние поступившие значения элементов данных
Data collection → Hosts	Управление хостами
Data collection → Templates	Управление шаблонами
Data collection → Maintenance	Режимы обслуживания
Alerts → Actions	Настройка оповещений

Раздел	Назначение
Reports → Availability report	Отчёты о доступности

Задание для самостоятельного изучения: пройдитесь по каждому пункту меню, опишите в отчёте, какие функции доступны в разделах Monitoring и Data collection.

---

6. Создание простого шаблона мониторинга

Задание 1. Создание шаблона

1. Перейдите в **Data collection → Templates**.
2. Нажмите кнопку **Create template**.
3. Заполните поля:
   - **Template name:** `Template Simple Linux Monitoring`
   - **Visible name:** `Simple Linux Monitoring`
   - **Template group:** выберите `Templates/Operating systems` (или создайте новую группу `Templates/Lab`).
4. Нажмите **Add**.

Задание 2. Добавление элементов данных (Items)

1. В списке шаблонов найдите созданный шаблон и нажмите на ссылку **Items** напротив него.
2. Нажмите **Create item** и добавьте три элемента данных:

Элемент 1 — Загрузка CPU:

Параметр	Значение
Name	`CPU load (1 min average)`
Type	`Zabbix agent`
Key	`system.cpu.load[all,avg1]`
Type of information	`Numeric (float)`
Update interval	`30s`
History storage period	`7d`
Trends storage period	`90d`

Элемент 2 — Свободная память:

Параметр	Значение
Name	`Available memory`
Type	`Zabbix agent`
Key	`vm.memory.size[available]`
Type of information	`Numeric (unsigned)`
Units	`B`
Update interval	`30s`

Элемент 3 — Свободное место на диске `/`:

Параметр	Значение
Name	`Free disk space on /`
Type	`Zabbix agent`
Key	`vfs.fs.size[/,pfree]`
Type of information	`Numeric (float)`
Units	`%`
Update interval	`60s`

3. После добавления всех трёх элементов нажмите **Update**.

Задание 3. Добавление триггеров (Triggers)

1. Вернитесь в список шаблонов и нажмите **Triggers** напротив созданного шаблона.
2. Нажмите **Create trigger** и создайте три триггера:

Триггер 1 — Высокая загрузка CPU:

Параметр	Значение
Name	`CPU load is too high on {HOST.NAME}`
Severity	`Warning`
Expression	`avg(/Template Simple Linux Monitoring/system.cpu.load[all,avg1],3m)>2`

Триггер 2 — Мало свободной памяти:

Параметр	Значение
Name	`Available memory is low on {HOST.NAME}`
Severity	`Average`
Expression	`last(/Template Simple Linux Monitoring/vm.memory.size[available])<100M`

Триггер 3 — Мало свободного места на диске:

Параметр	Значение
Name	`Free disk space is low on {HOST.NAME}`
Severity	`High`
Expression	`last(/Template Simple Linux Monitoring/vfs.fs.size[/,pfree])<10`

3. Сохраните каждый триггер.

Задание 4. Добавление графика (Graph)

1. В шаблоне перейдите в раздел **Graphs** и нажмите **Create graph**.
2. Заполните:
   - **Name:** `CPU Load and Memory`
   - **Graph type:** `Normal`
3. Добавьте два элемента на график:
   - `CPU load (1 min average)` — цвет зелёный;
   - `Available memory` — цвет синий.
4. Нажмите **Add**, затем **Update**.

---

7. Добавление хоста и привязка шаблона

Задание 5. Добавление хоста

1. Перейдите в **Data collection** → **Hosts**.
2. Нажмите **Create host**.
3. Заполните поля:
   - **Host name:** `Zabbix server`
   - **Visible name:** `Zabbix Server (Docker)`
   - **Host group:** выберите `Linux servers` (или создайте группу `Lab Hosts`).

4. В блоке **Interfaces** нажмите **Add** → **Agent** и укажите:
   - **IP address:** `zabbix-agent` (имя сервиса в Docker-сети) или `127.0.0.1`, если агент проброшен на хост.
   - **Port:** `10050`
   - **Connect to:** `DNS` (`zabbix-agent`).
5. В блоке **Templates** начните вводить название `Simple Linux Monitoring` в поле **Link new templates**. Выберите созданный шаблон из выпадающего списка.
6. Нажмите **Add**, затем **Add** (или **Update**).

Задание 6. Проверка мониторинга

1. Через 1–2 минуты перейдите в **Monitoring** → **Latest data**.
2. Отфильтруйте данные по хосту `Zabbix Server (Docker)`. Убедитесь, что появились значения по трём элементам данных:
   - CPU load (1 min average)
   - Available memory
   - Free disk space on /
3. Перейдите в **Monitoring** → **Hosts**. Убедитесь, что у хоста горит зелёный индикатор доступности (ZBX).
4. Перейдите в **Monitoring** → **Graphs** и выберите график `CPU Load and Memory` для созданного хоста.

Задание 7. Проверка срабатывания триггера

1. Запустите на виртуальной машине нагрузку на CPU (в отдельном терминале):

```
# Установите stress-ng, если его нет
sudo apt update && sudo apt install -y stress-ng

# Создайте нагрузку на 2 ядра на 5 минут
stress-ng --cpu 2 --timeout 300s
```

2. Пока нагрузка выполняется, перейдите в **Monitoring** → **Problems**. Через 3 минуты (согласно выражению триггера `avg(...,3m)>2`) должна появиться проблема `CPU load is too high on Zabbix Server (Docker)`.
3. Дождитесь окончания нагрузки и убедитесь, что проблема исчезла (статус `RESOLVED`).
4. Сделайте скриншоты графика CPU и списка проблем для отчёта.

Задания для самостоятельного выполнения

Вариант 1

1. Добавьте в шаблон элемент данных для мониторинга количества процессов: ключ `proc.num[]`.
2. Создайте триггер: срабатывает, если количество процессов меньше 50.
3. Привяжите обновлённый шаблон к хосту и проверьте поступление данных.

Вариант 2

1. Добавьте в шаблон элемент данных для мониторинга сетевого трафика на интерфейсе `eth0`: ключ `net.if.in[eth0]`.
2. Создайте график зависимости входящего трафика от времени.
3. Запустите скачивание большого файла (`wget`) и пронаблюдайте рост графика.

Вариант 3

1. Создайте второй шаблон `Template Simple Linux Monitoring Extended` на основе первого, добавив элемент `system.uptime` (время работы системы).
2. Создайте триггер: срабатывает, если система перезагружалась (uptime < 10 минут).
3. Привяжите новый шаблон к хосту вместо старого. Убедитесь, что данные собираются корректно.

Завершение работы

После выполнения всех заданий необходимо освободить ресурсы для этого выполните команду:

```
docker compose down -v
```

Контрольные вопросы

1. Какие компоненты входят в состав Zabbix? Опишите их назначение.
2. Что такое элемент данных (Item)? Какие типы элементов данных существуют?
3. Что такое шаблон (Template) и зачем он нужен? Как привязать шаблон к хосту?
4. Какие порты используются Zabbix Server и Zabbix Agent по умолчанию?
5. Как проверить, что агент успешно подключился к серверу и данные поступают?

Требования к отчёту

1. **Тема и цель работы.**
2. **Содержимое файла** `compose.yaml` (полный текст).
3. **Скриншоты:**
   - список запущенных контейнеров (`docker compose ps`);
   - главная страница веб-интерфейса Zabbix после входа;
   - созданный шаблон со списком элементов данных;
   - список триггеров шаблона;
   - добавленный хост с привязанным шаблоном;
   - страница **Latest data** с поступившими значениями;
   - график `CPU Load and Memory`;
   - страница **Problems** со сработавшим триггером (и со статусом RESOLVED после устранения).
4. **Результаты выполнения индивидуального варианта.**
5. **Выводы:** что освоено, какие трудности возникли, какие навыки приобретены.

Приложение. Полезные команды

```
# Запуск стека
docker compose up -d

# Остановка стека (с сохранением данных)
docker compose stop

# Остановка и удаление контейнеров (данные в томе сохраняются)
docker compose down

# Полное удаление с данными
docker compose down -v

# Просмотр логов сервера
docker compose logs -f zabbix-server

# Просмотр логов агента
docker compose logs -f zabbix-agent
```

```
# Подключение к контейнеру агента для диагностики
docker exec -it zabbix-agent bash
```

Важно: при удалении тома `postgres-data` все данные мониторинга будут потеряны. Для повторного запуска стека после `docker compose down -v` потребуется заново настроить Zabbix (создать хост, шаблон и т.д.), поэтому рекомендуется использовать `docker compose stop` вместо `down` при завершении работы.